

Step 1 — A Minimal Server With One Tool — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("notes-server")
notes = []

@mcp.tool()
def add_note(text):
    notes.append(text)
    return f"Note added. There are now {len(notes)} note(s)."
```

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

This works. It's missing type hints, a real docstring, config/logging, and any thought about stdout discipline — all fine for a first pass that just proves the connection works, but worth adding once you know the Inspector can actually call it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with a real description

```
# server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("notes-server")
notes: list[str] = []

@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note to the local notes store. Returns a short confirmation."""
    notes.append(text)
    return f"Note added. There are now {len(notes)} note(s)."
```

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Expected behavior: `mcp dev server.py` opens the Inspector; calling `add_note` with `{"text": "Buy milk"}` returns `"Note added. There are now 1 note(s)."`. Calling it a second time in the same session returns `"...now 2 note(s)."`. Closing and reopening the connection (a fresh process) resets the count to 0 — this version really is only in-memory.

Approach 2 — config and logging wired in from the start

```
# server.py
from mcp.server.fastmcp import FastMCP
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

mcp = FastMCP("notes-server")
notes: list[str] = []
```

```
@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note to the local notes store. Returns a short confirmation."""
    notes.append(text)
    logger.info("Note added (in-memory, count=%d)", len(notes))
    return f"Note added. There are now {len(notes)} note(s)."
```



```
if __name__ == "__main__":
    logger.info("Starting notes-server over stdio")
    mcp.run(transport="stdio")
```

Difference from Basic: full type hints on `add_note`, a docstring that reads like a real tool description (Doc06's "a tool's description is really a prompt" idea, applied here even though there's only one tool and no ambiguity yet), and Approach 2 wires in the same config/logger pattern every project in this curriculum starts with — `logger.info(...)` instead of a `print()`, which matters more here than usual, since stdout is the protocol channel on the stdio transport.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a minimal client script, instead of the Inspector

```
# client_step1_check.py -- a throwaway script, just to prove the connection
# without relying on the Inspector's UI. Not part of the final project files.
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await session.list_tools()
            print("Tools this server offers:", [t.name for t in tools.tools])

            result = await session.call_tool("add_note", {"text": "Buy milk"})
            print("Result:", result)

asyncio.run(main())
```

Expected output:

```
Tools this server offers: ['add_note']
Result: <CallToolResult with the confirmation text inside it>
```

This is the exact shape Step 4 grows into a full client — worth running once here, even briefly, so the Inspector doesn't stay a black box.

Approach 2 — hardened against a stray `print()` breaking the protocol

```
# server.py
from mcp.server.fastmcp import FastMCP
from config import load_config
```

```

from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

# NEVER call print() anywhere in this file or any module it imports --
# on the stdio transport, stdout IS the protocol channel. A stray print()
# here corrupts every message the client tries to read after it.

mcp = FastMCP("notes-server")
notes: list[str] = []

@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note to the local notes store. Use this whenever the user
    wants to save, remember, or jot down a piece of text. Returns a short
    confirmation once the note is stored."""
    notes.append(text)
    logger.info("Note added (in-memory, count=%d)", len(notes))
    return f"Note added. There are now {len(notes)} note(s)."

if __name__ == "__main__":
    logger.info("Starting notes-server over stdio")
    mcp.run(transport="stdio")

```

Expected behavior: identical to Intermediate Approach 2 from the client's point of view. The difference only shows up when something goes wrong elsewhere in the codebase — a teammate (or future-you) adding a debug `print()` to `db.py` in Step 2 would silently break the whole server's connection, in a way that's confusing to diagnose unless you already know `stdout` is off-limits. Writing that constraint down as a comment, right where the server starts, is cheap insurance against exactly that bug.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the server works, tested by hand through the Inspector. Approach 1 proves the same thing with real client code instead of a UI — the actual mechanism Step 4 builds on. Approach 2 doesn't change behavior at all; it documents, in the one file every future change to this project touches, the one constraint (no `print()`, ever) that's easy to violate by accident and expensive to debug once violated.

Which one should you actually write? Intermediate Approach 2 is what belongs in `server.py` for the rest of this project — it's small, correct, and already has config/logging wired in. Do run Advanced Approach 1's client script once, by hand, even though it's not one of this project's final files

— seeing a real `ClientSession` work, this early, makes Step 4 feel like "the thing I already did once, now for real" instead of new material.